

SQL PROJECT

Title: Amazon sales project

Subtitle: Data-Driven Insights for E-Commerce

Name: Kulkarni Vaishnavi

INTRODUCTION

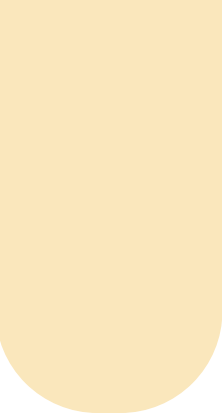
Project Objective:

To use SQL to analyze orders, product sales, shipping data, and customer behavior, and generate useful business insights that could support decision-making in an e-commerce environment.

Goal:

To build a hands-on, business-focused SQL portfolio that demonstrates:

- The ability to write clean, efficient SQL
- Problem-solving using multi-table datasets
- Answering real business questions



TOOLS & TECHNOLOGIES

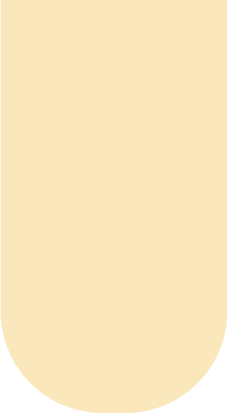
- **Database:** MySQL
- **Language:** SQL
- **Skills Demonstrated:**
 - Data Filtering, Grouping
 - Joins (INNER, LEFT)
 - Aggregation (SUM, COUNT, AVG)
 - Window Functions (RANK, ROW_NUMBER, LAG)
 - CTEs & Subqueries



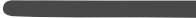
DATASET OVERVIEW

Tables Used:

- orders: Contains order-level details like date and customer ID
- order_items: Line-item level detail for each order
- products: Includes product name and category
- shipping: Covers shipment details like city, state, service, and fulfillment



KEY CHALLENGES

- 
- 
- Cleaning missing values (postal code and fulfillment)
 - Aggregating data by month, category, and region
 - Performing time-based analysis (rolling, MoM)
 - Delivering clear output for business understanding
- 
- 

How many total orders were placed.

This query counts the number of unique orders in the dataset.

```
SELECT
    COUNT(DISTINCT order_id) AS total_orders
FROM
    orders;
```

	total_orders
▶	68876

💡 Insight:

Gives a quick snapshot of total customer transactions — a basic metric for sales volume.

What is the total revenue generated.

Calculates total revenue by multiplying quantity by price for each product and summing it all.

```
SELECT
    SUM(qty * amount) AS total_revenue
FROM
    order_items;
```

	total_revenue
▶	71617820.00

💡 Insight:

Shows how much the company earned from product sales — essential for tracking growth and performance.

List all unique shipping service levels used.

Pulls all distinct shipping options from the shipping table.

```
SELECT DISTINCT  
    ship_service  
FROM  
    shipping;
```

	ship_service
▶	Standard
	Expedited

💡 Insight:

Understanding available service levels helps evaluate logistics diversity and support delivery cost analysis.

Which cities have the highest number of orders. (Top 5 cities)

Counts the number of orders per city and shows the top 5 with the highest volume.

```
SELECT
    s.ship_city, COUNT(o.order_id) AS total_orders
FROM
    orders o
    JOIN
    shipping s ON o.order_id = s.order_id
GROUP BY s.ship_city
ORDER BY total_orders DESC
LIMIT 5;
```

	ship_city	total_orders
►	BENGALURU	5947
	HYDERABAD	4276
	MUMBAI	3562
	NEW DELHI	3245
	CHENNAI	2966

💡 Insight:
Helps the business target high-performing regions for better marketing and delivery strategies.

Find the monthly total revenue.

Groups revenue by month using order date.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,  
       SUM(qty * amount) AS total_revenue  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
GROUP BY month  
ORDER BY month;
```

	month	total_revenue
▶	2022-03	86935.00
	2022-04	24435263.00
	2022-05	15546884.00

💡 Insight:

Tracks monthly sales performance, helping to detect growth, dips, or seasonal trends.

Identify the top 3 best-selling products by quantity.

Ranks products based on total units sold.

```
select p.category as category, count(ot.qty) as quantity from products as p
join order_items as ot
on p.sku = ot.sku
group by category
order by quantity desc
limit 3;
```

	category	quantity
▶	Set	41754
	kurta	39841
	Western Dress	13052

💡 Insight:

Identifies top performers — helpful for promotions and inventory prioritization.

Which product categories contribute the most to total revenue

Groups revenue by product category.

```
SELECT p.category,  
       SUM(ot.qty * ot.amount) AS category_revenue  
FROM order_items ot  
JOIN products p ON ot.sku = p.sku  
GROUP BY p.category  
ORDER BY category_revenue DESC;
```

	category	category_revenue
▶	Set	36141884.00
	kurta	18910348.00
	Western Dress	10256117.00
	Top	4939988.00
	Ethnic Dress	703738.00
	Blouse	424874.00
	Bottom	131963.00
	Saree	108603.00
	Dupatta	305.00

💡 Insight:
Reveals category-level contribution to overall sales — key for product line decisions.

Calculate cumulative monthly revenue over time.

Calculates cumulative revenue month by month.

```
WITH monthly_revenue AS (  
    SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,  
           SUM(oi.qty * oi.amount) AS revenue  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    GROUP BY month  
)  
SELECT month,  
       revenue,  
       SUM(revenue) OVER (ORDER BY month) AS cumulative_revenue  
FROM monthly_revenue;
```

	month	revenue	cumulative_revenue
▶	2022-03	86935.00	86935.00
	2022-04	24435263.00	24522198.00
	2022-05	15546884.00	40069082.00



Insight:

Visualizes long-term sales growth, helping measure compounding impact and pace of revenue generation.

Calculate 7-day rolling sum of daily revenue.

Uses a window function to calculate revenue for a 7-day rolling period.

```
WITH monthly_revenue AS (  
    SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,  
           SUM(oi.qty * oi.amount) AS revenue  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    GROUP BY month  
)  
SELECT month,  
       revenue,  
       SUM(revenue) OVER (ORDER BY month) AS cumulative_revenue  
FROM monthly_revenue;
```

	order_date	revenue	rolling_7_day_revenue
▶	2022-03-31	86935.00	86935.00
	2022-04-01	729834.00	816769.00
	2022-04-02	771324.00	1588093.00
	2022-04-03	846189.00	2434282.00
	2022-04-04	768971.00	3203253.00
	2022-04-05	807696.00	4010949.00
	2022-04-06	767624.00	4778573.00
	2022-04-07	770107.00	5461745.00
	2022-04-08	862881.00	5594792.00
	2022-04-09	828092.00	5651560.00
	2022-04-10	920075.00	5725446.00
	2022-04-11	850446.00	5806921.00
	2022-04-12	760900.00	5760125.00
	2022-04-13	850029.00	5842530.00
	2022-04-14	943225.00	6015648.00

💡 Insight: Helps monitor short-term performance trends to respond quickly to revenue fluctuations.

What percentage of revenue from each city contributes to its state's total revenue

Calculates each city's revenue and shows its percentage contribution within its state.

```
WITH city_revenue AS (  
    SELECT ship_state, ship_city,  
           SUM(oi.qty * oi.amount) AS city_revenue  
    FROM shipping s  
    JOIN orders o ON s.order_id = o.order_id  
    JOIN order_items oi ON o.order_id = oi.order_id  
    GROUP BY ship_state, ship_city  
)  
SELECT ship_state, ship_city, city_revenue,  
       ROUND(100.0 * city_revenue / SUM(city_revenue) OVER (PARTITION BY ship_state), 2) AS city_pct_of_state  
FROM city_revenue  
ORDER BY ship_state, city_pct_of_state DESC;
```

	ship_state	ship_city	city_revenue	city_pct_of_state
▶	ANDAMAN & NICOBAR	Port Blair	49427.00	66.82
	ANDAMAN & NICOBAR	PROTHRAPUR	6773.00	9.16
	ANDAMAN & NICOBAR	Portblair	3183.00	4.30
	ANDAMAN & NICOBAR	FERRARGUNJ	3125.00	4.22
	ANDAMAN & NICOBAR	BAMBOOFLAT	2968.00	4.01
	ANDAMAN & NICOBAR	Haddo	1334.00	1.80
	ANDAMAN & NICOBAR	SOUTH ANDAMAN	1299.00	1.76
	ANDAMAN & NICOBAR	School line	1199.00	1.62
	ANDAMAN & NICOBAR	Prem nagar , Port Blair	1129.00	1.53
	ANDAMAN & NICOBAR	Andaman and nicobar islands port Blair	999.00	1.35
	ANDAMAN & NICOBAR	PORT BLAIR / South Andaman	854.00	1.15
	ANDAMAN & NICOBAR	Port Blair , South Andaman	518.00	0.70
	ANDAMAN & NICOBAR	SWARAJ DWEET	459.00	0.62
	ANDAMAN & NICOBAR	Goalghar port blair	399.00	0.54
	ANDAMAN & NICOBAR	GARACHARMA	307.00	0.42
	ANDHRA PRADESH	VISAKHAPATNAM	231119.00	15.21
	ANDHRA PRADESH	VIJAYAWADA	122578.00	8.06
	ANDHRA PRADESH	GUNTUR	112052.00	7.37
	ANDHRA PRADESH	Nellore	85327.00	5.61



Insight:

Shows which cities drive the most revenue within each state — valuable for local strategy and expansion focus.

Calculate Month-over-Month (MoM) % change in revenue.

Uses LAG() to calculate percentage change in revenue from the previous month.

```
WITH monthly_revenue AS (  
    SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,  
           SUM(oi.qty * oi.amount) AS revenue  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    GROUP BY month  
)  
SELECT month, revenue,  
       LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,  
       ROUND(((revenue - LAG(revenue) OVER (ORDER BY month)) /  
             LAG(revenue) OVER (ORDER BY month)) * 100, 2) AS mom_change_pct  
FROM monthly_revenue;
```

	month	revenue	prev_month_revenue	mom_change_pct
▶	2022-03	86935.00	NULL	NULL
	2022-04	24435263.00	86935.00	28007.51
	2022-05	15546884.00	24435263.00	-36.38

💡 Insight:
Reveals growth trends, seasonality, and effectiveness of campaigns or changes.

Get the top 3 best-selling products by revenue in each month.

Ranks products by monthly revenue using window functions.

```
with monthly_revenue as (  
  select month(o.order_date) as month, p.category as category, sum(ot.qty * ot.amount) as revenue,  
  row_number() over(partition by month(o.order_date) order by sum(qty * amount) desc) as rnk from orders as o  
  join order_items as ot on o.order_id = ot.order_id  
  join products as p on ot.sku = p.sku  
  group by month, category  
)  
select month, category, revenue from monthly_revenue  
where rnk <= 3  
order by month, rnk;
```

	month	category	revenue
▶	3	Set	47557.00
	3	kurta	28009.00
	3	Western Dress	5479.00
	4	Set	13257544.00
	4	kurta	6592776.00
	4	Western Dress	2503142.00
	5	Set	7199300.00
	5	kurta	3737076.00
	5	Western Dress	2977556.00

💡 Insight:
Helps managers identify monthly product leaders, guiding stock planning and promotional priorities.

CONCLUSION

- This project helped me strengthen my SQL skills by applying them to solve real-world e-commerce problems.
- I learned how to write clean, efficient queries, handle messy data, and generate business insights that matter.
- From tracking revenue trends to analyzing customer and shipping behavior, I turned raw data into meaningful decisions — just like I would in a real analytics role.